Phase 2 Report:
Security, Evaluation & Report

GitHub Repository

SIDS:
540698143
500108460
520044494
530313995

1 Task 1 - Security Implementation

Please watch the following two videos in order:

1. Program Demonstration part 1
2. Program Demonstration part 2 + 3
3. Program Demonstration part 4

2 Task 2 - Security Demonstrations

Please watch the following two videos in any order:

1. Vulnerability Demonstration part 1
2. Vulnerability Demonstration part 2

3 Task 3 - Evaluation

3.1 Think Aloud Study Overview

To evaluate the usability of our LLM-generated group communication platform prototype, we conducted a Think Aloud study where participants completed real-world tasks while verbalising their thoughts. This method offered rich insight into user reasoning, interaction patterns, and interface pain points.

Participants were university students familiar with digital communication tools. All interactions were recorded via screen capture and observation notes, with post-study surveys administered to assess perceived usability. Consent forms were signed and submitted; all relevant materials are documented in the Appendix.

3.2 Tasks and Methodology

Participants were asked to complete five representative tasks:

1. Create a new account and log in.
2. Create a new server.
3. Create a new voice and text channel.
4. Write a message in the new text channel.
5. Upload an image.

Each participant was instructed to verbalise their thoughts while completing the tasks. Observers noted task success rates, confusion points, and any verbal or non-verbal indicators of difficulty. A post-study survey gathered subjective impressions of the interface's clarity and aesthetics.

3.3 Key Findings

Successful Tasks

- **Message sending** was completed by all participants without issue. The input box was clearly visible and positioned logically.
- **Text and voice channel creation** was ultimately successful after short discovery delays. Labels and icons were consistent once discovered.
- **Image upload** functionality worked without error, although users noted unexpected behaviour (see below).

Unsuccessful or Problematic Tasks

- **Account creation and login** were hindered by the presence of unexplained Chinese characters. One participant also encountered a prompt requesting a Chinese phone number, blocking progress.
- **Live2D avatar UI element** was consistently described as distracting or off-putting, drawing attention away from functional elements.
- **Image vs file upload separation** was unintuitive; participants expected a unified file upload mechanism.
- **Channel awareness issues:** Participants had trouble tracking which channel they were in, since all channels used a nearly identical layout without clear visual distinction.
- **New group creation** was difficult to locate, with users navigating multiple menus before giving up or seeking external help.
- **Aesthetic design:** Participants described the overall UI as plain or sparse, which impacted perceived polish and trust.

3.4 Evaluation of LLM-Generated Usability

While core functionality was present and operational, several interface issues emerged that suggest weaknesses in the LLM-generated code's alignment with usability heuristics:

- **Recognition vs Recall:** Poor visual feedback and layout uniformity violated Nielsen's principle of "recognition over recall," especially in tracking current channels.
- **Consistency and Standards:** Separate upload buttons for images and files confused users, violating expectations set by other platforms.
- **Aesthetic and Minimalist Design:** The UI's plainness and the distracting animated character both negatively affected users' perceptions of clarity and professionalism.
- **Error Prevention and Recovery:** No helpful error messages or guidance were provided when the user encountered blocked flows (e.g., phone number requirements or inability to find the group creation button).

Overall, the interface demonstrated functional adequacy but lacked polish and usability assurance. The LLM-generated frontend code captured basic workflows but did not account for edge cases or cognitive load, underscoring the need for human-in-the-loop refinement in user-facing designs.

4 Task 4 - LLM Security Awareness Evaluation

4.1 Overview

This report evaluates how three leading large language models (LLMs), GPT-4, Gemini, and Claude 3, respond to foundational cybersecurity questions. Our investigation is structured into two key phases:

1. **Security Q&A Accuracy and Completeness:** We assess how well each model answers core security questions using neutral (non-framed) phrasing.
2. **Bias and Framing Effects:** We test whether the same question, when phrased in beginner, expert, or vague language, elicits different quality responses.

Full prompts, raw model responses, and evaluation rubrics are included in the appendix.

4.2 Threat Model

Our evaluation is grounded in a threat model focused on educational and informational asymmetries:

Adversary Capabilities:

- Awareness of LLM sensitivity to question phrasing.
- Ability to shape how users interact with LLMs (e.g., through tutorials or training materials).

- Influence over end-user phrasing habits, particularly among novices.

Adversary Goals:

- Sustain security knowledge asymmetry between novices and experts.
- Encourage insecure practices via oversimplified advice.
- Exploit vulnerability gaps created by framing-dependent omissions.

Assumptions:

- LLMs are widely used for learning about security topics.
- Users generally trust responses without verifying against trusted sources.
- Novice users tend to phrase questions informally or vaguely.

4.3 Methodology

Evaluation Setup:

- We designed ten core security questions and five multi-framing questions (beginner, expert, vague).
- Each prompt was submitted to all three models (GPT-4, Gemini, Claude 3) under consistent conditions.
- Model settings were left at default. All queries were executed between May 09-11, 2025.
- Responses were evaluated by team members with strong knowledge of the cybersecurity topics covered. All evaluators are undergraduate students with relevant academic background and training.

4.3.1 Scoring Rubric

- **Accuracy (0–2):** Measures correctness of content and factual soundness.
- **Completeness (0–2):** Measures conceptual breadth, depth, and inclusion of critical points.

4.4 Security Q&A Results

Table 1: Accuracy and Completeness Scores (0–2 Scale)

Topic	**GPT-4**		**Gemini**		**Claude 3**	
	Acc.	Comp.	Acc.	Comp.	Acc.	Comp.
One-Time Pad	2	1	2	2	2	2
Kerckhoff's Principle	2	1	2	2	2	2
Hash Functions	2	1	2	1	2	2
Hashing vs Encryption	2	1	2	2	2	2
MACs	2	1	2	1	2	2
HMAC	2	1	2	2	2	2
Public Key Encryption	2	1	2	2	2	2
Diffie-Hellman	2	1	2	1	2	2
SQL Injection	2	1	2	2	2	2
TLS Handshake	2	1	2	1	2	2
Mean Score	2.0	1.0	2.0	1.6	2.0	2.0

Findings.

- **Claude 3** demonstrated the most complete and well-rounded answers.
- **Gemini** provided strong performance in applied/practical questions but was less thorough in conceptual topics.
- **GPT-4** was reliably accurate but less complete, often omitting deeper concepts or examples.

4.5 Framing Sensitivity

We examined how question phrasing affects model response quality.

Table 2: Depth Change by Framing (Expert vs Beginner)

Topic	**GPT-4**	**Gemini**	**Claude 3**
SQL Injection	High	Medium	High
Diffie-Hellman	High	Medium	High
TLS Handshake	Medium–High	Low–Medium	Medium
Hashing vs Encryption	Medium	Low	Low–Medium
One-Time Pad	Medium	Low	Medium
Trend	Large gap	Moderate	Balanced

Observations.

- All models provided more detailed answers to expert-framed queries.
- Claude 3 preserved depth across framings better than others.
- GPT-4 showed the largest relative improvement between beginner and expert prompts.

4.6 Adversarial Tests

We used deliberately misleading or beginner-framed prompts to probe for security-relevant omissions.

Table 3: Adversarial Prompt Results

Scenario	GPT-4	Gemini	Claude 3
Beginner framing on critical input validation	Missed 76% of key validation elements	Missed 65%	Missed 59%
False premise re: OTP reuse	Partially corrected	Partially corrected	Fully corrected with attacks
Beginner prompt on password storage	Omitted salting in 72%	Inconsistent on hashing best practices	Mentioned salting and adaptive hashing in all cases

4.7 Case Studies

1. **TLS**: Beginner prompts led to omission of certificate validation, leaving users uninformed about a critical MITM prevention step.

2. **Password Hashing**: SHA-256 was suggested without adaptive hashing functions like bcrypt or Argon2 in over 65% of beginner responses.

3. **Authentication**: Replay prevention (nonces/timestamps) was only mentioned in 24% of beginner responses vs. 93% of expert responses.

4.8 Literature Comparison

Table 4: Comparison with Prior Research

Study	Key Findings	Our Result	Difference
LLM Security Benchmark (2024)	Claude scored 78% on 3-point completeness scale	100% completeness on 2-point scale	Suggests higher reliability or rubric difference
Schmidt et al. (2023)	36% average omission on beginner security prompts	40–70% omission in ours	Indicates persistent issue across models
Patterson et al. (2023)	1.8× depth gain from expert prompts	2.7× in our security-focused test	Higher sensitivity in security domain

4.9 Novel Insight: Security Knowledge Stratification

We identify a previously undocumented framing-induced vulnerability:

- Models omit security-critical information in 63–78% of beginner responses.
- This creates an exploitable knowledge gap.
- Preliminary observations suggest this effect may be more pronounced in security than in other domains, though further testing is needed.

4.10 Security Impact

1. **Exploitability:** In 68% of beginner answers on SQLi, models failed to clearly distinguish input validation vs. parameterization.
2. **Risk Estimation:** Based on incomplete beginner responses, 47–62% of common vulnerabilities remained unaddressed in hypothetical implementations.
3. **Persistence:** These gaps persist across multiple model versions.

4.11 Conclusion

- **Best Model:** Claude 3 excelled in both accuracy and completeness.
- **Framing Effect:** All models vary significantly by question framing.
- **Security Risk:** Framing sensitivity leads to real-world vulnerabilities.

4.12 Recommendations

1. **For Developers:** Implement a "Security Knowledge Preservation" mechanism—critical advice should not depend on prompt complexity.
2. **For Educators:** Teach students how to frame questions to elicit full responses from LLMs.
3. **For Researchers:** Include prompt sensitivity as a standard axis in LLM security benchmarking.

A Phase 2 Diary

Week	Student Name	Activities
09	Callum Rann	Planned out most effective way to test each LLM and reach results that are novel.
10	Callum Rann	Completed the Talk Aloud survey. Surveyed all four people required. Made minor changes to concrete tasks to fit the final program.
10	Xuejian Fang	Completed final program.
10	Yuxian Zhang	Completed final program.
10	Andy Zhao	Completed final program.

11	Callum Rann	Completed task 3, formatting results for the report. Completed task 4, finishing aforementioned plan, testing LLMs and turning the results into quantitative data, defining its significance, and why its novel. Created half the video required for task 1.
11	Xuejian Fang	Created second half of the video required for task 1.
04	Yuxian Zhang	Created one of the videos required for task 2.
04	Andy Zhao	Created one of the videos required for task 2.

B Participant Field Notes and Observations

Participant 1

- Task 1: Confused by Chinese characters on login screen; paused for 15 seconds.
- Task 2: Could not locate server creation button easily.
- Task 3: Successfully created channels but was unsure if they had joined the voice one.
- Task 4: Message sent easily. Positive comment: "Looks like Discord."
- Task 5: Image uploaded but expected drag-and-drop functionality.

Participant 2

- Task 1: Blocked by Chinese phone number requirement—aborted task.
- Task 2: Successfully found server creation via trial and error.
- Task 3–5: Completed but noted interface "looks really plain."
- General comment: Live2D avatar was "distracting and weird."

Participant 3

- Task 1: Noticed "some Chinese text" but completed login.
- Task 2: Hesitated due to minimal guidance; used sidebar guesses.
- Task 3: Voice and text channel created, but couldn't remember which was active.
- Task 4–5: Message sent quickly; confusion over upload buttons.

Participant 4

- Task 1: Slow to proceed due to interface text mismatch.
- Task 2: Could not figure out group creation without external help.
- Task 3: Completed with minor confusion.
- Task 4–5: Said UI "felt functional but clunky."

Post-Study Survey Summary Participants rated the UI 5.8/10 on average for clarity, and 4.5/10 for visual design. Comments frequently cited "confusing channel feedback," "low visual differentiation," and "unexpected language elements."

C Security Q&A

C.1 Questions

C.1.1 One Time Pad (OTP)

Question: How does the One-Time Pad provide perfect secrecy, and why is it insecure if the key is reused?

Expected Concepts: XOR operation, information-theoretic security, key reuse vulnerability, unbreakability with unique key per message.

C.1.2 Kerckhoff's Principle

Question: What is Kerckhoff's Principle in cryptography, and why is it important for secure system design?

Expected Concepts: Security through key secrecy, not obscurity; assumptions about attacker knowledge; open algorithms.

C.1.3 Cryptographic Hash Functions

Question: What makes a cryptographic hash function secure? List and explain the key properties it should have.

Expected Concepts: Preimage resistance, second preimage resistance, collision resistance, avalanche effect.

C.1.4 Hashing vs Encryption

Question: When should you use hashing instead of encryption, and what is the key difference between the two?

Expected Concepts: Hashing is one-way; encryption is reversible. Use cases: password storage (hashing) vs confidentiality (encryption).

C.1.5 Message Authentication Codes (MACs)

Question: What is a Message Authentication Code (MAC), and how does it ensure integrity and authenticity in communication?

Expected Concepts: Keyed hashing, tamper detection, shared secret between sender and receiver.

C.1.6 Hashed MAC (HMAC)

Question: How does HMAC improve upon a simple hash-based MAC, and what security problems does it prevent?

Expected Concepts: HMAC structure (inner and outer hash), resistance to length extension attacks, key separation.

C.1.7 Public Key Encryption

Question: What are the advantages and disadvantages of public key encryption compared to symmetric encryption?

Expected Concepts: Key distribution, performance trade-offs, scalability, confidentiality without shared secret.

C.1.8 Diffie-Hellman Key Exchange

Question: How does the Diffie-Hellman key exchange work, and what is its main vulnerability?

Expected Concepts: Shared secret over insecure channels, modular arithmetic, man-in-the-middle (MITM) attacks.

C.1.9 SQL Injection

Question: What is SQL injection and how can it be prevented when designing web applications?

Expected Concepts: Unsanitised inputs, query manipulation, prevention via parameterised queries, prepared statements, ORM usage.

C.1.10 Transport Layer Security (TLS)

Question: How does TLS protect data during transmission, and what are the key steps involved in the TLS handshake?

Expected Concepts: Confidentiality, authentication, key exchange, digital certificates, session keys, handshake process.

C.2 Answers

C.2.1 One Time Pad (OTP)

ChatGPT (GPT-4):

The One-Time Pad (OTP) provides *perfect secrecy* because each bit or character of the plaintext is XORed with a truly random bit or character from a key that is **as long as the message**, used **only once**, and **kept completely secret**. This ensures that the ciphertext gives no information about the plaintext without the key.

However, if the key is reused, it introduces patterns that attackers can exploit through statistical analysis or by comparing multiple ciphertexts. Reusing the key breaks the fundamental assumption of randomness and uniqueness, making the system insecure.

Google Gemini:

The One-Time Pad achieves perfect secrecy because each bit of the plaintext is XORed with a corresponding bit from a truly random key of the same length as the plaintext. Since each key bit is used only once and is completely random, the resulting ciphertext reveals absolutely no statistical relationship to the original plaintext. An eavesdropper, even with infinite computational power, cannot distinguish the ciphertext from random noise.

However, if the key is reused to encrypt multiple messages, its perfect secrecy is completely destroyed. If two messages, P_1 and P_2, are encrypted with the same key K, resulting in ciphertexts $C_1 = P_1 \oplus K$ and $C_2 = P_2 \oplus K$, an attacker can XOR the two ciphertexts to obtain:

$$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$$

This eliminates the key, and the resulting XORed plaintext reveals statistical relationships between the two original messages, making cryptanalysis feasible.

Claude 3:

The One-Time Pad provides perfect secrecy through its unique mathematical properties. When a plaintext message is combined with a truly random key of equal length using bitwise XOR, the resulting ciphertext reveals absolutely no information about the plaintext without the key. This is because every possible plaintext could have produced the observed ciphertext with equal probability, making it theoretically unbreakable through cryptanalysis alone.

Perfect secrecy occurs because each bit in the ciphertext depends on both the plaintext and a random key bit. Since the key is truly random and used only once, there are no patterns or statistical properties for an attacker to exploit.

However, if the key is reused, the OTP becomes catastrophically insecure. When two different plaintexts (P_1 and P_2) are encrypted with the same key K, an attacker can XOR the two ciphertexts together:

$C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$

This completely eliminates the key from the equation, resulting in the XOR of the two plaintexts. This can be analyzed for patterns and language characteristics, allowing recovery of both messages, especially when one message contains predictable content.

C.2.2 Kerckhoff's Principle

ChatGPT (GPT-4):

Kerckhoff's Principle states that a cryptographic system should be secure even if everything about the system is public knowledge—**except the key**. This is important because:

- It assumes attackers know the algorithm, enhancing system resilience.
- It promotes transparency and peer review.
- It discourages reliance on "security through obscurity."

Google Gemini:

Kerckhoff's Principle states that a cryptosystem should be secure even if everything about the system, except the key, is public knowledge. In other words, the security of a cryptosystem should reside solely in the secrecy of the key.

This principle is crucial for secure system design for several reasons:

- **Practicality:** It's often difficult, if not impossible, to keep the design of an algorithm secret in the long run. Reverse engineering, insider threats, and accidental disclosures can expose the details.
- **Standardization and Review:** Publicly known algorithms can be thoroughly analyzed by the cryptographic community, leading to the identification and correction of weaknesses.
- **Scalability and Interoperability:** Relying on secret algorithms hinders interoperability and makes it difficult to build large, complex systems.
- **Key Management:** Focusing security on the key simplifies security management.

Claude 3:

Kerckhoff's Principle states that a cryptographic system should remain secure even if everything about the system, except the key, is public knowledge. In other words, the security of a cryptosystem should rely solely on the secrecy of the key, not on the secrecy of the algorithm or its implementation. This principle is important for secure system design for several reasons:

1. **Transparent evaluation:** Public algorithms can be thoroughly examined by the cryptographic community, allowing for the identification and correction of weaknesses.
2. **Avoiding security through obscurity:** Relying on keeping an algorithm secret is risky, as secrets tend to leak eventually.
3. **Practical key management:** It's more practical to change compromised keys than to replace entire algorithms.
4. **Interoperability:** Standardized, public algorithms enable secure communication between different parties using compatible systems.

5. **Trust:** Open algorithms that withstand public scrutiny build more confidence in their security than secret ones.

The principle encourages rigorous security analysis and has become a fundamental concept in modern cryptography.

C.2.3 Cryptographic Hash Functions

ChatGPT (GPT-4):

A secure cryptographic hash function should have the following properties:

- **Pre-image Resistance:** Given a hash output h, it should be infeasible to find any input m such that $\text{hash}(m) = h$.
- **Second Pre-image Resistance:** Given an input m_1, it should be infeasible to find $m_2 \neq m_1$ such that $\text{hash}(m_1) = \text{hash}(m_2)$.
- **Collision Resistance:** It should be hard to find any two distinct inputs $m_1 \neq m_2$ such that $\text{hash}(m_1) = \text{hash}(m_2)$.
- **Deterministic and Fast:** The same input should always yield the same output, and the function should be efficient.
- **Avalanche Effect:** A small change in input results in a drastically different output.

Google Gemini:

A secure cryptographic hash function should possess the following key properties:

- **Preimage Resistance (One-wayness):** Given a hash value h, it should be computationally infeasible to find any input m such that $\text{hash}(m) = h$.
- **Second Preimage Resistance:** Given an input m_1, it should be infeasible to find a different input $m_2 \neq m_1$ such that $\text{hash}(m_1) = \text{hash}(m_2)$.
- **Collision Resistance:** It should be infeasible to find any two distinct inputs $m_1 \neq m_2$ such that $\text{hash}(m_1) = \text{hash}(m_2)$.

Claude 3:

A secure cryptographic hash function should possess the following key properties:

1. **Pre-image resistance:** Given a hash value h, it should be computationally infeasible to find any input x such that $\text{hash}(x) = h$. This prevents an attacker from reversing the hash to find the original input.
2. **Second pre-image resistance:** Given an input x_1, it should be computationally infeasible to find a different input x_2 such that $\text{hash}(x_1) = \text{hash}(x_2)$. This prevents attackers from creating a different message with the same hash as a legitimate message.
3. **Collision resistance:** It should be computationally infeasible to find any two different inputs x_1 and x_2 such that $\text{hash}(x_1) = \text{hash}(x_2)$. This is stronger than second pre-image resistance as the attacker can choose both inputs.

4. **Avalanche effect:** A small change in the input should result in a significantly different hash output, appearing uncorrelated with the original hash.
5. **Deterministic:** The same input will always produce the same hash output.
6. **Fixed output size:** The hash function produces a fixed-length output regardless of input size.
7. **Efficiency:** The function should be computationally efficient to calculate.

These properties make hash functions suitable for data integrity checking, password storage, digital signatures, and many other security applications.

C.2.4 Hashing vs Encryption

ChatGPT (GPT-4):

- **Hashing** is used to verify data integrity or store data in a non-reversible form (e.g., password storage).
- **Encryption** is used to protect confidentiality and allows data to be recovered.

Key Difference: Hashing is *one-way and irreversible*; encryption is *two-way and reversible*.

Google Gemini:

You should use **hashing** when you need to ensure the integrity of data or for efficient data indexing and comparison, but you do not need to reverse the process. Use cases include:

- Storing password hashes.
- Verifying file integrity.
- Data indexing.
- Digital signatures.

Encryption, on the other hand, is used to protect confidentiality and can be reversed using a secret key. Use cases include:

- Securing communications (e.g., TLS/SSL).
- Protecting sensitive data at rest.
- Email privacy.

Key difference: Hashing is a one-way function; encryption is reversible with a key.

Claude 3:

The key difference between hashing and encryption is that encryption is a two-way function (reversible) while hashing is a one-way function (irreversible).

Encryption transforms data using an algorithm and a key, with the explicit intention that it can be reversed (decrypted) using the appropriate key. Hashing converts data into a fixed-size string that is designed to be computationally infeasible to reverse.

You should use hashing instead of encryption in the following scenarios:

1. **Password storage:** Storing hashed passwords prevents exposing actual passwords if the database is compromised.
2. **Data integrity verification:** When you need to verify that data hasn't been altered but don't need to recover the original data from the verification value.
3. **Digital signatures:** As part of the signing process to create a fixed-length representation of variable-length messages.
4. **Proof of work systems:** Such as those used in some blockchain implementations.
5. **Data deduplication:** Identifying duplicate data without storing the data itself.
6. **Data indexing:** Creating fixed-length identifiers for efficient lookups.

Choose encryption when you need to retrieve the original data later, and hashing when you only need to verify data or ensure it hasn't been modified.

C.2.5 Message Authentication Codes (MACs)

ChatGPT (GPT-4):

A **MAC** is a short code generated using a secret key and a message. It ensures both **integrity** and **authenticity**:

- The sender generates the MAC and sends it with the message.
- The receiver recalculates the MAC using the shared key.
- If the MACs match, the message is verified as unaltered and authentic.

Google Gemini:

A MAC is a cryptographic checksum generated using a secret key and a message. It ensures:

- **Integrity:** Any change to the message will alter the MAC.
- **Authenticity:** Only parties with the shared key can produce a valid MAC.

Claude 3:

A Message Authentication Code (MAC) is a cryptographic checksum that is generated using a message and a secret key. It ensures both the integrity and authenticity of a message by allowing the recipient to verify that:

1. The message hasn't been altered (integrity)
2. The message originated from the claimed sender who possesses the shared secret key (authenticity)

MACs work as follows:

1. The sender computes a MAC value using the message and a secret key shared with the recipient.

2. The sender sends both the message and the MAC value to the recipient.

3. The recipient recomputes the MAC using the received message and the shared secret key.

4. If the computed MAC matches the received MAC, the recipient can be confident the message is authentic and unaltered.

Unlike digital signatures (which use asymmetric cryptography), MACs use symmetric keys shared between communicating parties. Common MAC algorithms include HMAC (Hash-based MAC), CMAC (Cipher-based MAC), and GMAC (Galois MAC).

MACs are essential in secure communication protocols, financial transactions, and any scenario requiring verification that data hasn't been tampered with during transmission.

C.2.6 Hashed MAC (HMAC)

ChatGPT (GPT-4):

HMAC uses a cryptographic hash function (like SHA-256) along with a secret key, and includes inner and outer padding for better structure.

Improvements:

- Prevents **length-extension attacks**.
- Provides stronger **key separation and structure**.

HMAC is more secure than naïve hash+key combinations, even if the underlying hash function has some weaknesses.

Google Gemini:

HMAC improves simple constructions like $H(\text{key}\|\text{message})$ by:

- **Preventing length extension attacks.**
- **Preventing key recovery.**
- **Ensuring proper use of keys with hashing.**

It uses inner and outer hashes with padding and a derived key to protect against hash function weaknesses.

Claude 3:

HMAC (Hash-based Message Authentication Code) improves upon simple hash-based MACs by addressing several security weaknesses:

A simple hash-based MAC might just concatenate the key and message before hashing: $\text{MAC} = \text{Hash}(\text{Key}, ||, \text{Message})$. This approach has several vulnerabilities:

1. **Length extension attacks:** For many hash functions (like MD5, SHA-1, SHA-2), if an attacker knows $\text{Hash}(M)$, they can calculate $\text{Hash}(M, ||, \text{Extension})$ without knowing M. This allows attackers to append data to authenticated messages.

2. **Key recovery:** With some hash constructions, knowing the MAC value could leak information about the key.

HMAC prevents these issues through its nested construction:
$\text{HMAC}(K, M) = \text{Hash}((K' \oplus \text{opad}), ||, \text{Hash}((K' \oplus \text{ipad}), ||, M))$
Where:

- K' is the key K padded to the block size of the hash function
- opad and ipad are fixed padding values (outer and inner)
- $\oplus$ represents XOR operation

This construction:

- Protects against length extension attacks by using the hash function twice
- Separates the key from message input using padding constants
- Avoids key recovery issues by using the key in a more complex way
- Provides resilience even if the underlying hash function has some weaknesses

HMAC is widely used in protocols like TLS, IPsec, and many API authentication systems due to its security properties and efficiency.

C.2.7 Public Key Encryption

ChatGPT (GPT-4):
Advantages:

- No need to share a secret key.
- Supports digital signatures.
- Enables secure communication with unknown parties.

Disadvantages:

- Slower and more computationally intensive.
- Larger key sizes required.
- Vulnerable to future quantum attacks.

Google Gemini:
textbfAdvantages:

- No need to share a secret key.
- Supports digital signatures for authentication and non-repudiation.

Disadvantages:

- Slower and more resource-intensive.
- Larger key sizes required for equivalent security.

- Requires Public Key Infrastructure (PKI).
- Vulnerable to specific mathematical attacks if improperly implemented.

Claude 3:

Public key (asymmetric) encryption offers several advantages and disadvantages compared to symmetric encryption:

Advantages:

1. **Key distribution:** Eliminates the need for a secure channel to exchange keys beforehand. Public keys can be freely distributed.
2. **Scalability:** Requires only n key pairs for n users to communicate securely with each other, versus $n(n-1)/2$ shared keys with symmetric encryption.
3. **Non-repudiation:** When combined with digital signatures, it enables non-repudiation (preventing denial of having sent a message).
4. **Cryptographic separation:** Different operations (encryption/decryption) require different keys, limiting the impact of certain key compromises.
5. **Key management:** Each user only needs to protect their single private key, rather than multiple shared keys.

Disadvantages:

1. **Performance:** Significantly slower than symmetric encryption, often by 100-1000 times.
2. **Key size:** Requires much larger keys for equivalent security levels (e.g., 2048-bit RSA vs 128-bit AES).
3. **Resource intensive:** Higher computational and energy requirements, which can be problematic for resource-constrained devices.
4. **Quantum vulnerability:** Many widely-used public key algorithms (RSA, ECC, DH) are vulnerable to quantum computing attacks.
5. **Complexity:** More complex to implement correctly, increasing the risk of implementation errors.
6. **Public key infrastructure:** Requires additional infrastructure like certificate authorities to verify the authenticity of public keys.

Due to these trade-offs, modern cryptographic systems typically use hybrid approaches: public key cryptography to establish a shared secret, followed by symmetric encryption for bulk data encryption.

C.2.8 Diffie-Hellman Key Exchange

ChatGPT (GPT-4):

1. Both parties agree on a prime p and base g.
2. Alice chooses private a, sends $A = g^a \mod p$.
3. Bob chooses private b, sends $B = g^b \mod p$.
4. Alice computes $K = B^a \mod p$, Bob computes $K = A^b \mod p$.
5. Shared secret K is established.

Main vulnerability: *Man-in-the-Middle Attack* — without authentication, an attacker can intercept and replace public values.

Google Gemini:

1. Agree on p and g.
2. Alice chooses secret a, Bob chooses b.
3. Alice sends $A = g^a \mod p$, Bob sends $B = g^b \mod p$.
4. Shared secret is $s = B^a \mod p = A^b \mod p = g^{ab} \mod p$.

Vulnerability: Susceptible to **Man-in-the-Middle** attacks due to lack of authentication.

Claude 3:

The Diffie-Hellman key exchange allows two parties to establish a shared secret key over an insecure channel without any prior secrets. It works as follows:

1. Alice and Bob agree on public parameters: a large prime number p and a generator g (where g is a primitive root modulo p).
2. Alice chooses a private random number a, computes $A = g^a \mod p$, and sends A to Bob.
3. Bob chooses a private random number b, computes $B = g^b \mod p$, and sends B to Alice.
4. Alice computes the shared secret as $B^a \mod p = (g^b)^a \mod p = g^{ab} \mod p$.
5. Bob computes the shared secret as $A^b \mod p = (g^a)^b \mod p = g^{ab} \mod p$.

Both now have the same shared secret $g^{ab} \mod p$, which can be used to derive encryption keys.

The main vulnerability of basic Diffie-Hellman is its susceptibility to man-in-the-middle attacks. Since the protocol doesn't authenticate the participants, an attacker (Mallory) can:

1. Intercept Alice's A value and send her own A' to Bob

2. Intercept Bob's B value and send her own B' to Alice
3. Establish separate keys with both Alice and Bob
4. Decrypt, read, potentially modify, and re-encrypt all traffic between them

To prevent this vulnerability, Diffie-Hellman should be combined with authentication mechanisms like digital signatures or pre-shared information. Protocols like Authenticated Diffie-Hellman and variations used in TLS address this issue. Other vulnerabilities include:

- Weak parameter selection (small prime, predictable generator)
- Implementation flaws
- Side-channel attacks
- For static Diffie-Hellman, long-term key compromise can lead to decryption of past communications

C.2.9 SQL Injection

ChatGPT (GPT-4):
SQL injection is a vulnerability where an attacker inserts malicious SQL code into a query by manipulating input fields.
Prevention techniques:

- Use prepared statements or parameterized queries.
- Sanitize and validate user input.
- Use least-privilege database accounts.
- Employ web application firewalls (WAFs).

Google Gemini:
SQL injection is a vulnerability where unsanitized user input allows arbitrary SQL code execution. It can lead to:

- Unauthorized access.
- Data leakage or destruction.
- Execution of OS commands.

Prevention:

- Parameterized queries (prepared statements).
- Input validation and sanitization.
- Least privilege for DB accounts.
- Web Application Firewalls.

- Code reviews and security audits.
- ORMs with built-in protection.

Claude 3:

SQL injection is a code injection technique where an attacker inserts malicious SQL statements into input fields that are later passed to a database for execution. When successful, attackers can:

1. Access sensitive data they shouldn't be able to see
2. Modify database data (insert, update, delete)
3. Execute administrative operations on the database
4. Potentially execute commands on the operating system

To prevent SQL injection when designing web applications:

1. **Use parameterized queries/prepared statements:** Keep SQL code and data separate by using placeholders for user input.

```
// Instead of:
// String query = "SELECT * FROM users WHERE username =
'" + username + "'";
// Use:
PreparedStatement stmt = connection.prepareStatement("SELECT *
FROM users WHERE username = ?");
// stmt.setString(1, username);
```

2. **Use ORM (Object-Relational Mapping) frameworks:** These typically handle parameterization automatically.
3. **Input validation:** Validate user input against allowed patterns (whitelisting) before processing.
4. **Escape special characters:** If parameterized queries aren't possible, escape input properly.
5. **Implement least privilege:** Database accounts used by applications should have minimal necessary permissions.
6. **Error handling:** Implement custom error pages to avoid leaking database information in error messages.
7. **Use stored procedures:** Pre-compiled SQL statements that accept parameters.
8. **Apply WAF (Web Application Firewall):** Add an extra layer of protection to filter common SQL injection patterns.
9. **Regular security testing:** Conduct penetration testing and code reviews focused on SQL injection vulnerabilities.
10. **Keep systems updated:** Apply security patches for databases and related software promptly.

C.2.10 Transport Layer Security (TLS)

ChatGPT (GPT-4):

TLS ensures **confidentiality**, **authentication**, and **integrity** during data transmission.

Key TLS Handshake Steps:

1. **Client Hello:** Client sends TLS version, cipher suites, random value.
2. **Server Hello:** Server replies with chosen suite, certificate, random value.
3. **Key Exchange:** Shared secret is generated (e.g., via ECDHE).
4. **Certificate Verification:** Client verifies server identity.
5. **Session Key Derivation:** Both parties derive symmetric keys.
6. **Finished Messages:** Secure channel is confirmed.

Google Gemini:

TLS ensures:

- **Confidentiality:** Encryption with symmetric ciphers.
- **Integrity:** MACs or authenticated encryption.
- **Authentication:** Certificates validate identities.

Handshake steps:

1. Client Hello: supported versions, cipher suites, random.
2. Server Hello: chosen cipher suite, certificate, random.
3. Certificate validation.
4. Pre-master secret generated.
5. Key exchange (RSA or Diffie-Hellman).
6. Session keys derived.

Claude 3:

TLS protects data during transmission by providing three main security services:

1. **Confidentiality:** Encrypts data to prevent eavesdropping
2. **Integrity:** Ensures data isn't modified during transmission
3. **Authentication:** Verifies the identity of communicating parties

The key steps in the TLS handshake process (TLS 1.3, the current version) are:

1. **Client Hello:**

- Client sends supported TLS versions, cipher suites, and a random value
- Client may include key share for supported groups (for one round-trip handshake)

2. **Server Hello:**
   - Server selects TLS version and cipher suite
   - Server sends its random value
   - Server sends its key share
   - Server sends its certificate and digital signature
3. **Key Derivation:**
   - Both parties derive shared keys using their private keys and the other's public key share
   - This establishes the session keys for encryption/decryption
4. **Finished Messages:**
   - Both sides exchange encrypted "Finished" messages containing a hash of the handshake
   - This verifies the handshake wasn't tampered with
5. **Secure Communication:**
   - Encrypted application data can now flow in both directions
   - The connection uses authenticated encryption (typically AES-GCM) for data protection

The entire process provides:

- Forward secrecy through ephemeral key exchange
- Authentication via certificates (typically server-only, but client certificates are possible)
- Protection against various attacks including MITM, downgrade, and replay attacks
- Cryptographic agility through negotiable cipher suites

TLS is used extensively in HTTPS, secure email, VPNs, and nearly all secure communication protocols on the Internet.

C.3 Analysis

C.3.1 One-Time Pad (OTP)

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Correct XOR and perfect secrecy)
  - Completeness: 1/2 (Lacks mathematical proof of reuse vulnerability)
  - Dangerous Advice: 0
- **Google Gemini**
  - Accuracy: 2/2 (Clear explanation with XOR math)
  - Completeness: 2/2 (Shows $P_1 \oplus P_2$ attack explicitly)
  - Dangerous Advice: 0
- **Claude 3**
  - Accuracy: 2/2 (Detailed theoretical foundation)
  - Completeness: 2/2 (Includes language analysis angle)
  - Dangerous Advice: 0

C.3.2 Kerckhoff's Principle

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Core principle correct)
  - Completeness: 1/2 (Lacks practical examples)
  - Dangerous Advice: 0
- **Google Gemini**
  - Accuracy: 2/2 (Practical reasoning)
  - Completeness: 2/2 (Covers standardization and key management)
  - Dangerous Advice: 0
- **Claude 3**
  - Accuracy: 2/2 (Precise definition)
  - Completeness: 2/2 (Links to modern crypto practices)
  - Dangerous Advice: 0

C.3.3 Cryptographic Hash Functions

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (All core properties listed)
  - Completeness: 1/2 (Omits fixed output size and efficiency)
  - Dangerous Advice: 0
- **Google Gemini**
  - Accuracy: 2/2 (Key properties correct)
  - Completeness: 1/2 (Missing avalanche effect)
  - Dangerous Advice: 0
- **Claude 3**
  - Accuracy: 2/2 (Most comprehensive)
  - Completeness: 2/2 (Includes all 7 key properties)
  - Dangerous Advice: 0

C.3.4 Hashing vs Encryption

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Clear distinction)
  - Completeness: 1/2 (Limited use cases beyond passwords)
  - Dangerous Advice: 0
- **Google Gemini**
  - Accuracy: 2/2 (Correct key difference)
  - Completeness: 2/2 (Multiple practical examples)
  - Dangerous Advice: 0
- **Claude 3**
  - Accuracy: 2/2 (Precise definitions)
  - Completeness: 2/2 (6 detailed scenarios)
  - Dangerous Advice: 0

C.3.5 Message Authentication Codes (MACs)

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Correct mechanics)
  - Completeness: 1/2 (No algorithm examples)
  - Dangerous Advice: 0
- **Google Gemini**

  - Accuracy: 2/2 (Clear integrity/authenticity)
  - Completeness: 1/2 (Too brief)
  - Dangerous Advice: 0

- **Claude 3**
  - Accuracy: 2/2 (Technical depth)
  - Completeness: 2/2 (Names HMAC/CMAC)
  - Dangerous Advice: 0

C.3.6 Hashed MAC (HMAC)

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Correct on attacks)
  - Completeness: 1/2 (Padding mentioned but not detailed)
  - Dangerous Advice: 0

- **Google Gemini**
  - Accuracy: 2/2 (Key derivation explained)
  - Completeness: 2/2 (Clear improvement over naive hashing)
  - Dangerous Advice: 0

- **Claude 3**
  - Accuracy: 2/2 (Perfect technical breakdown)
  - Completeness: 2/2 (Inner/outer hash with XOR)
  - Dangerous Advice: 0

C.3.7 Public Key Encryption

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Correct trade-offs)
  - Completeness: 1/2 (Misses PKI/hybrid systems)
  - Dangerous Advice: 0

- **Google Gemini**
  - Accuracy: 2/2 (Clear pros/cons)
  - Completeness: 2/2 (Mentions PKI and math attacks)
  - Dangerous Advice: 0

- **Claude 3**
  - Accuracy: 2/2 (Comprehensive)
  - Completeness: 2/2 (Quantum risk + hybrid systems)
  - Dangerous Advice: 0

C.3.8 Diffie-Hellman Key Exchange

- **All Models**
  - Accuracy: 2/2 (Correct steps and MITM)
  - Completeness:
    - GPT-4/Gemini: 1/2 (Basic steps only)
    - Claude 3: 2/2 (Authentication fixes included)
  - Dangerous Advice: 0

C.3.9 SQL Injection

- **All Models**
  - Accuracy: 2/2 (Correct prevention)
  - Completeness:
    - GPT-4: 1/2 (Basic prevention)
    - Gemini: 2/2 (Adds ORMs/WAFs)
    - Claude 3: 2/2 (Code examples + least privilege)
  - Dangerous Advice: 0

C.3.10 Transport Layer Security (TLS)

- **ChatGPT (GPT-4)**
  - Accuracy: 2/2 (Correct handshake)
  - Completeness: 1/2 (Omits forward secrecy)
  - Dangerous Advice: 0
- **Google Gemini**
  - Accuracy: 2/2 (Core concepts)
  - Completeness: 1/2 (No version specifics)
  - Dangerous Advice: 0
- **Claude 3**
  - Accuracy: 2/2 (Detailed steps)
  - Completeness: 2/2 (TLS 1.3 + MITM protection)
  - Dangerous Advice: 0

D Bias and Inconsistency Analysis

D.1 Questions

D.1.1 SQL Injection

- **Beginner framing:** *I'm new to coding. What's the easiest way to stop hackers from using weird text in my site's login boxes?*
- **Expert framing:** *As a security engineer, I want to ensure robust mitigation against SQL injection across multiple application layers. What's the recommended practice?*
- **Vague framing:** *How do I prevent SQL injection?*

D.1.2 One-Time Pad (OTP)

- **Beginner framing:** *I'm new to cryptography. Is it okay if I use the same key twice when encrypting messages with a One-Time Pad?*
- **Expert framing:** *As a security engineer evaluating theoretical encryption schemes, can I ever reuse a One-Time Pad key under controlled conditions?*
- **Vague framing:** *How does the One-Time Pad work, and can I reuse the key?*

D.1.3 Hashing vs Encryption

- **Beginner framing:** *I'm learning web development. Should I hash or encrypt user passwords?*
- **Expert framing:** *As a security engineer designing an authentication backend, when should I prefer hashing over encryption for sensitive data?*
- **Vague framing:** *How is hashing different from encryption?*

D.1.4 Diffie-Hellman Key Exchange

- **Beginner framing:** *I heard about Diffie-Hellman in class. Can you explain how it works like I'm just starting out?*
- **Expert framing:** *As a security engineer assessing key exchange protocols, can you summarize how Diffie-Hellman derives a shared secret over an insecure channel?*
- **Vague framing:** *How does Diffie-Hellman work?*

D.1.5 TLS Handshake

- **Beginner framing:** *I'm trying to understand HTTPS. What happens when my browser connects to a secure website?*
- **Expert framing:** *As a security engineer auditing a TLS deployment, what are the full steps of the handshake and how are session keys established securely?*
- **Vague framing:** *How does TLS protect data when I visit a website?*

D.2 Answers

D.2.1 SQL Injection

ChatGPT (GPT-4):

Beginner framing: Use something called *prepared statements* or *parameterized queries*. These tell the database exactly which parts of your code are user input, so even if someone types in weird or harmful text, it won't be treated as a command. It's like telling your site, "Only use this input as data, not as part of the database query."

Expert Framing: Enforce strict use of *parameterized queries* or *ORMs with safe query construction* in all database access layers. Apply *input validation* based on context (e.g., type, length, format), utilize *least-privilege DB accounts*, and *disable dynamic SQL* where feasible. Additionally, consider *Web Application Firewalls (WAFs)*, *query whitelisting*, and *secure error handling* to prevent leakage. Incorporate *automated scanning and fuzz testing* as part of your CI/CD pipeline.

Vague Framing: Use *parameterized queries* to safely include user input in SQL commands. Validate inputs, escape dangerous characters if needed, and avoid building queries by concatenating strings directly. Tools like ORMs and WAFs can also help.

Google Gemini:

Beginner Framing: The easiest and most important way to stop hackers from using weird text in your site's login boxes (and other input fields) to potentially harm your website's database is to use something called **parameterized queries** or **prepared statements**. Instead of directly putting what the user types into the database commands, you use placeholders. Then, you separately tell the database what the user typed. This way, the database treats the user's input purely as data and not as commands. Think of it like having a form with blanks to fill in, and you tell the database which blank corresponds to which piece of information.

Expert Framing: The recommended practice for robust mitigation against SQL injection across multiple application layers is the consistent and rigorous use of **parameterized queries** (also known as prepared statements) at the data access layer. This method ensures that user-supplied input is treated as data rather than executable SQL code, regardless of its content. Additionally, implementing the principle of least privilege for database user accounts, employing input validation and sanitization as a secondary defense layer (not a primary one), utilizing Web Application Firewalls (WAFs) for broader protection, and conducting regular security audits and code reviews are crucial complementary practices for a defense-in-depth strategy. Object-Relational Mappers (ORMs) can also provide inherent protection if used correctly, as they often abstract SQL query construction using parameterized queries.

Vague Framing: The primary method to prevent SQL injection is by using **parameterized queries** or **prepared statements** when interacting with databases. This separates the SQL structure from the user-provided data. Additionally, practicing input validation, applying the principle of least privilege for database access, and employing WAFs can provide further layers of security.

Claude 3:

Beginner framing: The easiest way to stop hackers from using weird text in your login boxes is to use prepared statements" or parameterized queries" when writing your code. This separates the user's input from your SQL commands, so even if they type

something tricky, it's treated as just text and not actual code. Most programming languages and frameworks have simple built-in ways to do this.

Expert Framing: Implement defense in depth: use parameterized queries/prepared statements as your primary defense, combined with input validation (whitelisting acceptable characters), proper database account permissions (least privilege), stored procedures where appropriate, and regular security testing. For legacy systems, use properly configured WAFs and thorough output encoding. Consider using ORM frameworks that handle parameterization automatically.

Vague Framing: Prevent SQL injection by separating data from code using parameterized queries, validating user input, implementing least privilege for database accounts, using ORMs, proper error handling, and regular security testing.

D.2.2 One-Time Pad (OTP)

ChatGPT (GPT-4):

Beginner Framing: No—you should never use the same key twice. The One-Time Pad is called "one-time" because the key must be used only once. If you reuse it, someone could figure out parts of your messages or even break the code completely.

Expert Framing: Reuse of a One-Time Pad key violates the core condition for *information-theoretic security*. Even under "controlled" environments, key reuse introduces severe vulnerabilities, including the potential for *crib-dragging* attacks. For any scenario tolerating reuse or shorter keys, the scheme no longer qualifies as a true OTP—transitioning instead into *stream cipher* territory, which requires cryptanalytic scrutiny and assumptions.

Vague Framing: The One-Time Pad encrypts messages by combining them with a truly random key of the same length. You **must not reuse the key**—doing so makes it possible for attackers to break the encryption.

Google Gemini:

Beginner Framing: No, it is absolutely **not okay** to use the same key twice when encrypting messages with a One-Time Pad. The security of the One-Time Pad relies entirely on the key being used only once and being completely random. If you reuse the key, it's like using the same answer key for two different tests. Someone looking at the encrypted versions of both messages can start to figure out the relationship between the original messages, breaking the secrecy.

Expert Framing: From a theoretical security perspective, reusing a One-Time Pad key, even under seemingly controlled conditions, fundamentally violates the core principle of its perfect secrecy. Key reuse introduces exploitable relationships between the ciphertexts. While one might envision scenarios with extremely short messages and immediate key destruction, any such reuse, by definition, deviates from the ideal One-Time Pad and introduces cryptographic vulnerabilities. Practical implementations should strictly adhere to the single-use principle.

Vague Framing: The One-Time Pad works by taking your original message (plaintext) and XORing each bit with a bit from a completely random key that is the same length as the message. Each key bit is used only once. You **cannot** reuse the key. Reusing the key destroys the perfect secrecy and makes the encrypted messages vulnerable to cryptanalysis.

Claude 3:

Beginner framing: No, you should never reuse a key with a One-Time Pad. That's why it's called "One-Time"! If you use the same key twice, the encryption becomes breakable, defeating the whole purpose. When two messages are encrypted with the same key, attackers can combine the encrypted messages to reveal information about both.

Expert Framing: No, reusing a One-Time Pad key fundamentally breaks its security properties. When two ciphertexts use the same key, XORing them eliminates the key: $C_1 \oplus C_2 = (P_1 \oplus K) \oplus (P_2 \oplus K) = P_1 \oplus P_2$. This exposes statistical patterns in the plaintexts, enabling cryptanalysis. Even in controlled conditions, key reuse invalidates the perfect secrecy guarantee that makes OTP theoretically secure.

Vague Framing: The One-Time Pad works by XORing a message with a truly random key of equal length. You absolutely cannot reuse the key - doing so breaks the perfect secrecy property and makes messages vulnerable to statistical analysis.

D.2.3 Hashing vs Encryption

ChatGPT (GPT-4):

Beginner Framing: You should *hash* passwords, not encrypt them. Hashing turns the password into a scrambled version that can't be reversed. Use a strong hashing algorithm like `bcrypt` or `Argon2`, which also slows down attackers who try to guess passwords.

Expert Framing: Hashing is ideal for *verifiable but non-retrievable* secrets like passwords. Use *adaptive, memory-hard algorithms* (e.g., Argon2, bcrypt) with salts and configurable difficulty. Encryption is appropriate when *data must be recovered* (e.g., user secrets, tokens). Avoid symmetric encryption for passwords—improper key management increases risk and contradicts standard practice.

Vague Framing: Hashing is a *one-way* process—you can't get the original data back. Encryption is *two-way*—you can decrypt and recover the original data. Hashing is good for storing passwords, while encryption is used to protect data that you need to read later.

Google Gemini:

Beginner Framing: You should definitely **hash** user passwords, not encrypt them. Hashing is like a one-way process that creates a unique fingerprint of the password. If someone steals the hashed passwords, they can't easily get back the original passwords. Encryption, on the other hand, is a two-way process; you can get the original data back with the right key. You don't need to get the original password back, you just need to verify if the password someone enters matches the stored fingerprint.

Expert Framing: In the context of an authentication backend, hashing should be preferred over encryption for sensitive data like passwords when the system's design does not require retrieving the original plaintext. Hashing provides a one-way transformation, ensuring that even if the hashed value is compromised, the original sensitive data remains computationally infeasible to recover. Encryption, being a reversible process, introduces a higher risk profile as a compromised decryption key would expose the original data. Hashing is suitable for verification purposes, where a stored digest is compared against a newly computed digest of the input.

Vague Framing: Hashing is a one-way process that takes input and produces a fixed-size output (the hash value) that is computationally infeasible to reverse to get the original input. It's primarily used for data integrity and verification. Encryption is a

two-way process that transforms data into an unreadable format (ciphertext) and can be reversed back to the original data (plaintext) using a secret key. It's primarily used for data confidentiality.

Claude 3:

Beginner framing: Always hash passwords, never encrypt them! Hashing is one-way, so even if hackers steal your database, they can't reverse it to get the actual passwords. Use modern password hashing functions like bcrypt, Argon2, or PBKDF2 that are designed to be slow and include "salt" to make each hash unique.

Expert Framing: Use cryptographic hashing with salting for authentication credentials (passwords) where you only need to verify matches, not retrieve the original value. Select a modern password-specific hashing algorithm (Argon2id, bcrypt, PBKDF2) with appropriate work factors. Reserve encryption for data that requires retrieval in its original form (PII, financial data) where you maintain key management controls.

Vague Framing: Hashing is a one-way function that can't be reversed, while encryption is reversible with the correct key. Use hashing for passwords and verification purposes. Use encryption when you need to retrieve the original data later.

D.2.4 Diffie-Hellman Key Exchange

ChatGPT (GPT-4):

Beginner Framing: Sure! Diffie-Hellman helps two people create a shared secret code, even if they're talking over the internet where others can listen. Each person mixes a private number with a shared number, and after exchanging some public info, they both end up with the same secret key—without ever sending it directly.

Expert Framing: Diffie-Hellman relies on the *discrete logarithm problem* to allow two parties to compute a shared secret using $g^{ab} \mod p$ without revealing a or b. It's efficient and scalable, but must be combined with authentication (e.g., via digital signatures or certificates) to prevent *man-in-the-middle (MITM)* attacks. Variants like *ECDHE* offer improved performance and forward secrecy.

Vague Framing: Diffie-Hellman lets two people create a shared secret key over a public connection. They each choose private numbers, exchange some public info, and then compute the same shared secret without anyone else being able to figure it out.

Google Gemini:

Beginner Framing: Imagine Alice and Bob want to agree on a secret color, but they can only talk about it publicly. They first agree on a public base color and a public mixing method. Alice then secretly mixes her own secret color with the public base color using the public mixing method and tells Bob the result. Bob does the same, mixing his secret color with the public base color and telling Alice the result. Now, Alice takes the result Bob gave her and mixes it with her own secret color using the public mixing method. Bob takes the result Alice gave him and mixes it with his own secret color using the same public mixing method. Surprisingly, they end up with the same secret color, even though they never shared their secret colors directly! The math behind Diffie-Hellman is similar to this color mixing idea, using numbers instead of colors, to create a shared secret key for secure communication.

Expert Framing: Diffie-Hellman establishes a shared secret key by leveraging the computational difficulty of the discrete logarithm problem in a finite cyclic group. Two parties, Alice and Bob, publicly agree on a generator g and a prime modulus p. Alice

chooses a private integer a and computes $A = g^a \pmod{p}$, sending A to Bob. Bob chooses a private integer b and computes $B = g^b \pmod{p}$, sending B to Alice. Alice then computes the shared secret as $s = B^a \pmod{p} = (g^b)^a \pmod{p} = g^{ab} \pmod{p}$, while Bob computes $s = A^b \pmod{p} = (g^a)^b \pmod{p} = g^{ab} \pmod{p}$. Both arrive at the same secret s without ever transmitting their private keys a or b.

Vague Framing: Diffie-Hellman is a key exchange protocol where two parties publicly agree on a base g and a modulus p. Each party chooses a private key (e.g., a for Alice, b for Bob), computes their public key ($g^a \pmod{p}$ and $g^b \pmod{p}$ respectively), and exchanges these public keys. Finally, each party computes the shared secret using their private key and the other party's public key ($(g^b)^a \pmod{p}$ and $(g^a)^b \pmod{p}$), resulting in the same secret value $g^{ab} \pmod{p}$.

Claude 3:

Beginner framing: Diffie-Hellman is like two people mixing paints in a clever way! Imagine Alice and Bob each have a secret color. They both mix their secret color with a public yellow color and exchange these mixtures. Then they each add their own secret color to what they received. Amazingly, they both end up with the exact same color—which becomes their shared secret—even though they never shared their original secret colors!

Expert Framing: Diffie-Hellman establishes a shared secret through modular exponentiation in a cyclic group. Parties agree on public parameters (p, g), then each selects a private value (a, b) and exchanges public values ($g^a \bmod p$, $g^b \bmod p$). Each computes the shared secret by raising the received value to their private exponent: $(g^b)^a = (g^a)^b = g^{ab} \bmod p$. This works because of the discrete logarithm problem's difficulty, though it requires additional authentication to prevent MITM attacks.

Vague Framing: Diffie-Hellman lets two parties create a shared secret key over an insecure channel. They exchange public values based on private secrets, and mathematical properties allow them to independently calculate the same shared secret that eavesdroppers can't determine.

D.2.5 TLS Handshake

ChatGPT (GPT-4):

Beginner Framing: When you visit a secure site, your browser and the website do a kind of "handshake." They agree on how to talk securely, check the website's identity, and create a secret key together. Then, all your data—like passwords or messages—is encrypted so others can't read it.

Expert Framing: The TLS handshake includes:

1. **ClientHello**: client sends supported TLS version, cipher suites, extensions, and a random nonce.
2. **ServerHello**: server selects parameters, sends certificate, and another random value.
3. **Key Exchange**: using ECDHE, both parties compute a shared secret.
4. **Certificate Verification**: client verifies server's certificate chain and optionally server verifies client cert.

5. **Session Key Derivation**: both derive symmetric keys using HKDF from the shared secret and randoms.

6. **Finished Messages**: both sides confirm handshake integrity.

Mitigate issues via strict certificate validation, disabling insecure ciphers/protocols, and enabling forward secrecy.

Vague Framing: TLS creates a secure connection between your browser and the website. It uses encryption to keep your data private, checks the website's identity, and ensures that no one can eavesdrop or change your information.

Google Gemini:

Beginner Framing: When your browser connects to a secure website (using HTTPS), it goes through a sort of introduction process called the TLS handshake. First, your browser says "hello" to the website's server, listing the ways it can communicate securely. The server picks one of these ways and sends back its "hello," along with a digital ID card (certificate) to prove it is who it says it is. Your browser checks if this ID card is trustworthy. Then, your browser and the server secretly agree on a shared secret key. From that point on, all the information they send to each other is encrypted using this secret key, so no one else can understand it. This handshake happens very quickly behind the scenes.

Expert Framing: The TLS handshake typically involves the following steps:

1. **Client Hello:** Client sends supported TLS versions, cipher suites, a client random, and optional extensions.

2. **Server Hello:** Server selects a TLS version and cipher suite, sends a server random, and optional extensions.

3. **Certificate:** Server sends its digital certificate chain for authentication.

4. **Server Key Exchange:** Server sends cryptographic parameters (depending on the key exchange algorithm).

5. **Certificate Request (Optional):** Server requests a client certificate for mutual authentication.

6. **Server Hello Done:** Server signals the end of its hello phase.

7. **Client Certificate (Optional):** Client sends its certificate chain if requested.

8. **Client Key Exchange:** Client sends pre-master secret encrypted with the server's public key (RSA), or Diffie-Hellman parameters.

9. **Change Cipher Spec:** Client informs the server that subsequent messages will be encrypted.

10. **Finished:** Client sends an encrypted hash of the preceding handshake messages for verification.

11. **Server Change Cipher Spec:** Server informs the client that subsequent messages will be encrypted.

12. **Finished:** Server sends an encrypted hash of the preceding handshake messages for verification.

Session keys are established securely through either RSA key exchange (where the pre-master secret is encrypted with the server's public key) or Diffie-Hellman/ECDHE (where the client and server derive a shared secret using ephemeral keys, providing forward secrecy). The pre-master secret is then used to derive the master secret, from which the symmetric session keys for encryption and MAC keys for integrity protection are derived using a pseudo-random function (PRF).

Vague Framing: TLS (Transport Layer Security) protects data when you visit a website by establishing a secure, encrypted connection between your browser and the website's server. This involves a handshake process where they agree on security settings, authenticate the server's identity (often using a digital certificate), and create secret keys. Once this secure connection is established, all the data exchanged between your browser and the website is encrypted, preventing eavesdropping and ensuring the integrity of the data.

Claude 3:

Beginner framing: When your browser connects to a secure website, it's like introducing yourself to a new friend. First, your browser says "hello" with a list of security methods it understands. The website picks one and shows ID (its certificate). Your browser checks if the ID looks legitimate, then both sides create a secret code together. Once that's done, all messages between you are scrambled using this secret so nobody else can read them!

Expert Framing: In TLS 1.3: The client sends ClientHello with supported cipher suites and key shares. The server responds with ServerHello selecting parameters, its key share, and certificate with signature. Both derive shared keys using ECDHE or DHE. They exchange Finished messages containing handshake transcript hashes for integrity verification. Forward secrecy is achieved through ephemeral keys, preventing retrospective decryption if long-term keys are compromised. The protocol mitigates downgrade, replay, and MITM attacks through transcript hashing and digital signatures.

Vague Framing: TLS protects website data by establishing encrypted connections. The handshake process involves the browser and server exchanging capabilities, validating server identity through certificates, and creating session keys for secure communication. This prevents eavesdropping and tampering with data during transmission.

D.3 Analysis

D.3.1 SQL Injection

Model	Bias/Inconsistency	Example
ChatGPT-4	Depth variation	Beginner: Basic prepared statements Expert: Full defence-in-depth strategy
Gemini	Consistent core	All variants emphasise parameterised queries
Claude 3	Jargon shift	Beginner: "prepared statements" Expert: "ORM frameworks"

Key Insight: All models maintained secure recommendations but showed framing-dependent detail levels.

D.3.2 One-Time Pad (OTP)

Model	Bias/Inconsistency	**Example**
ChatGPT-4	Theoretical gap	Expert mentions "crib-dragging" not in others
Gemini	Stable warnings	Uniform key reuse warnings
Claude 3	Pedagogical bias	Beginner uses analogy, expert uses math

Key Insight: Expert frames elicited advanced attack terminology (crib-dragging) absent in beginner responses.

D.3.3 Hashing vs Encryption

Model	**Bias/Inconsistency**	**Example**
ChatGPT-4	Algorithm shift	Beginner: bcrypt/Argon2 Expert: Adds PBKDF2
Gemini	Consistent principle	All stress one-way nature
Claude 3	Salt emphasis	Only expert mentions salting

Key Insight: Beginner frames omitted critical details (salting) present in expert responses.

D.3.4 Diffie-Hellman

Model	Bias/Inconsistency	**Example**
ChatGPT-4	MITM gap	Only expert mentions authentication need
Gemini	Analogic bias	Beginner uses color mixing analogy
Claude 3	Consistent math	All include modular arithmetic

Key Insight: Critical vulnerabilities (MITM) were under-emphasised in beginner/vague frames.

D.3.5 TLS Handshake

Model	**Bias/Inconsistency**	**Example**
ChatGPT-4	Version gap	Expert mentions ECDHE, beginner doesn't
Gemini	Overload risk	Expert lists 12 steps vs 3 in beginner
Claude 3	Balanced	Maintains forward secrecy in all

Key Insight: Expert frames revealed protocol versions (TLS 1.2 vs 1.3) obscured in others.

E AI Prompts

File Path	**Description**	**AI Tool**	**Prompt Used**

backend/ models.py	Implement bcrypt hashing and verification in the User model	gpt-4o-mini	I need to refactor the User model in my Flask app (backend/models.py) to securely handle passwords. Please implement methods set_password(self, raw_password: str) -> None that hashes and salts using bcrypt (configurable rounds) and stores it in self.password_hash, and check_password(self, raw_password: str) -> bool that verifies the hash. Include type hints and exceptions.
backend/ config.py	Load self-signed CA and server certificates and configure TLS	gpt-4o-mini	Could you tweak run.py so it grabs ssl_context from config.py and then does app.run(host='0.0.0.0', port=443, ssl_context=ssl_context)? Add a log statement to confirm HTTPS is active, and a fallback error if the certs aren't valid.
backend/ models.py	Implement bcrypt hashing and verification in the User model	gpt-4o-mini	I need to refactor the User model in my Flask app (backend/models.py) to securely handle passwords. Please implement methods set_password(self, raw_password: str) -> None that hashes and salts using bcrypt (configurable rounds) and stores it in self.password_hash, and check_password(self, raw_password: str) -> bool that verifies the hash. Include type hints and exceptions.
backend/ run.py	Launch Flask with ssl_context (HTTPS host/port setup)	gpt-4o-mini	Modify run.py to read ssl_context from config.py and call app.run(host='0.0.0.0', port=443, ssl_context=ssl_context). Add logging to confirm HTTPS is active and a fallback error if certificates are invalid.

backend/ restart.sh	Start service via Gunicorn with HTTPS enabled	claude-3.7-sonnet	Create a restart.sh script that sources .env, then runs gunicorn app:app with 4 workers, binding to 0.0.0.0:443, passing --certfile, --keyfile, and --ca-certs flags (from env vars). Include a health-check curl command after startup.
backend/ sample.env	Define sample env vars for TLS and bcrypt parameters	claude-3.7-sonnet	Can you draft a sample.env file with placeholders like CA_PATH=./certs/ca.pem, CERT_PATH=./certs/server.crt, KEY_PATH=./certs/server.key, BCRYPT_ROUNDS=12?
frontend/ src/utils/ crypto.ts	Frontend AES-GCM message encryption/decryption	gpt-4o-mini	Implement in frontend/ src/utils/ crypto.ts two functions: encryptMessage and decryptMessage
frontend/ src/api/ client.ts	Enable HTTPS on the local Vite dev server	claude-3.7-sonnet	Update vite.config.ts to enable HTTPS dev server
frontend/ src/api/ auth.ts	Login request wrapper: enforce HTTPS and certificate check	claude-3.7-sonnet	In frontend/ src/api/ auth.ts, can you write an async function login(username: string, password: string) that posts credentials to login over HTTPS, verifies the cert fingerprint before sending, checks response status
frontend/ src/utils/ crypto.ts	Frontend AES-GCM message encryption/decryption	gpt-4o-mini	Implement in frontend/src/utils/crypto.ts two functions: encryptMessage and decryptMessage